Potato

- **Time limit:** 10 minutes
- **Environment:** one GPU (≈16 GB VRAM), no internet
- **Solution size:** `solution.ipynb` ≤ 1 MB
- **Storage:** 5 GB

Task

Your friend suggests playing a guessing game. He, as the judge, picks one hidden word from a fixed vocabulary, and you must find it in at most 30 turns. Each turn the judge compares two words and reports which is semantically closer to the hidden word. Every game starts from the fixed pair `lamp vs potato`, because they are two of your friend's favorite things. Your program then proposes one new word. The winner of the comparison is kept and compared against your next proposal. You win a game the moment you propose the hidden word exactly. Matching is case-insensitive. Every word you propose must be in `dataset/vocabulary.json`.

There is a full example in `solution.ipynb` with protocol and data loading. You can change the PublicEmbeddingPlayer class. Your program is initialized once and plays every game in a single run; the protocol creates a fresh PublicEmbeddingPlayer at the start of each game.

The Judge

Your program sends one JSON object to the Judge and the Judge responds with one JSON object.

A worked example, with the hidden word shown only to explain the protocol:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp",   "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock",   "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock",   "word2": "hammer"}
-> {"new_word": "shovel"}                                    <- matches: game won
-> {"status": "win"}
```

Turns are indexed from 1 to 30.

The `verdict` options are `first` meaning that word1 is closer, `second` meaning that word2 is closer or `same` meaning that both words are equally close to the hidden word.

`winner_word` is the word retained for the next comparison. On a `same` verdict, the first word stays.

Dataset

Shared by every split:

- `dataset/vocabulary.json` — 1602 unique lowercase words. The hidden word is always one of these.
- `dataset/public_embeddings.npy` — `float32`, shape `(1602, 2560)`. Row `i` corresponds to word `i` in the vocabulary. These are *public* embeddings; the judge uses a different, private representation.

The splits are sets of hidden words:

Split	Words	Answers	Use it to
`test_public`	120	✅ `dataset/test_public.json`	run your solution and self-score
`test_leaderboard_a`	120	hidden	live leaderboard
`test_leaderboard_b`	120	hidden	final ranking

There is no `train` split — nothing is fitted from labelled rows.

Provided models

Two pretrained embedding models ship with the task and may be used:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card](https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card](https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Both must be loaded from their local path; a Hugging Face hub id such as `"BAAI/bge-m3"` triggers a download and fails, because judging is offline. Each directory contains a runnable `example.py` showing the offline call.

Available libraries: `numpy`, `torch`, `sentence-transformers`. No internet, no downloads, no other packages.

Output

None. This is an interactive task: your solution writes no answer file; it talks to the judge over stdin/stdout as described above.

Metric

A game found on turn `t` scores `1.0 - 0.02 × max(0, t - 10)`; a game not solved within 30 turns scores `0`. So turns 1–10 score `1.00`, turn 20 scores `0.80`, turn 30 scores `0.60`.

Your task score is the mean game score × 100, between `0.00` and `100.00`.

The 10-minute limit is a single budget covering start-up, preparation and all 120 games in the test set.

How to submit

1. Open `solution.ipynb`, edit `PublicEmbeddingPlayer`, and run all cells to make sure it's working.
2. Optionally, check it locally: `python local_test.py solution.ipynb --limit 5`. The local judge uses the *public* embeddings, so its score is only a guide.
3. Save `solution.ipynb`.
4. Open the Git tab in the left sidebar of JupyterLab.
5. Stage `solution.ipynb` (the **+** icon next to it).
6. Enter a commit message and click Commit.
7. Click the cloud-with-up-arrow to push.
8. Return to this Contest page and click Submit, with the commit message matching the one you've provided.

Submit exactly one file, named `solution.ipynb`, covering any necessary preparations and inference.